C Programing Tutorial - 2

C Syntax

Lets look at a final example.

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv){
        int i = 0;
        int res = 0;

        for (i=1;i<argc;i++){
                printf("%d\n",atoi(argv[i]));
                res+=atoi(argv[i]);
        }
        printf("%s%d\n", "result is: ", res);

        return 0;
}
```

Header file needed for the usage of “atoi” function.

Array of characters pointers listing all arguments passed. “argv[0]” is the program name.

Number of command line arguments passed including the name of the program.

Print the arguments passed after converting them to integers using the function “atoi”.

Print the sum of numbers passed as command line arguments. Multiple format specifiers for “printf” are used. “%s” used to print “result is” and “%d” is used to print the sum of numbers.

Exercise

- Write a C program
  - that takes two integers as command line arguments,
  - print all the passed arguments
  - and then prints their sum using a function **sum(int, int)**

Hint: don’t forget to check if two integers are passed as command line arguments

Exercise - Answer

```c
#include <stdio.h>
#include <stdlib.h>

int sum(int, int);
int main(int argc, char **argv){
        int i = 0;

        if( argc==3){
                for (i = 1; i < argc; i++){
                        printf("%d\n",atoi(argv[i]));
                }
                printf("%s%d\n", "result is: ", sum(atoi(argv[1]),atoi(argv[2])));
        }
        else{
                printf("%s\n","You entered wrong number of arguments you should have entered 2");
        }
        return 0;
}
int sum(int int1, int int2){
        return int1+int2;
}
```

User Input

```c
#include <stdio.h>

int main() {

  int age;
  printf("Enter your age\n");
  scanf("%d", &age);

  /* ... intensive and very carefull checking is needed ... */

  printf("Your age is %d\n", age);
  return 0;
}
```

Scan a number from the user

Exercise 3

1) Write a C program that scans a number “n” from the user and prints all even numbers from 1 to “n”

2) Write a C program that finds and prints the maximum between three numbers. **Scan** one number from the user and pass two numbers as **command line arguments**. You should write and use a function **find_max()** to find the maximum.

```c
#include <stdio.h>

int main(){
        int n;
        scanf("%d",&n);
        int i =1;
        for (i = 1; i <=n; i++)
                if(i%2==0)
                        printf("%d ",i);
        printf("\n");
        return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
int find_max(int,int,int);
int main (int argc, char **argv){
        if(argc !=3){
                printf("You should enter only 2 numbers as command line\n");
        }else{
                int i;
                scanf("%d",&i);
                int j = atoi(argv[1]);
                int k = atoi(argv[2]);
                printf("The max between %d, %d, and %d is %d\n",i,j,k,find_max(i,j,k));
        }
        return 0;
}
int find_max(int a, int b, int c){
        if(a > b){
                if( a > c)
                        return a;
                else
                        return c;
        }else if (b > c)
                return b;
        else
                return c;}
```

Revision Quiz ☺

- Write a C program that reads x , y and z integers from the command line and 2 integers i and j from the scanner.

  - Your program should check if the number of arguments entered by the user is correct and print an error message if the user entered more or less arguments.
  - Your program should print all the 5 integers in the below format: (assume input is 1 2 3 for x,y,z and 8 9 for i and j)

    x = 1

    y = 2

    z = 3

    i = 8

    j = 9
  - your program should declare a function “multi” that takes x y and j and returns their multiplication.
  - Your program should print the result of the function

Pointers

1. What are pointers?
    1. As the name implies, a pointer “points” to a location in memory. They are variables that store memory addresses.
2. For what purposes?
    1. Retrieving value stored in memory location
    2. Passing large objects without copying them
    3. Accessing dynamically allocated memory
    4. Referring to functions
3. Notations used with pointers
    1. **&** → address of operator
    2. * → dereference operator
4. Declaration of a pointer
    1. **<variable_type> *<name>;**

C Pointers

```c
#include <stdio.h>

int main () {

   int  var = 20;   /* variable declaration */
   int  *ip;        /* pointer variable declaration */
   ip = &var;  /* store address of var in pointer variable*/

   printf("Address of var variable: %x\n", &var  );

   /* address stored in pointer variable */
   printf("Address stored in ip variable: %x\n", ip );

   /* access the value using the pointer */
   printf("Value of *ip variable: %d\n", *ip );

   return 0;
}
```

Which are valid initialization

```
int a = 1;
int b = 2;
int *c = 3;
int *d = b;
int *e = &b;
int *f = *b;
```

WHY?
Pointers stores addresses not values

3 is a value

b is equal to 2 which is a value

b is not a pointer

What is the output

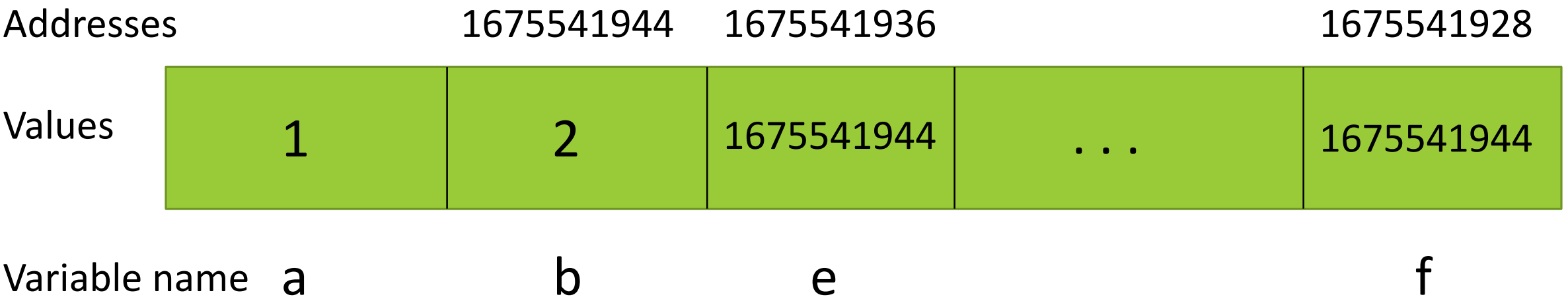
```
int a = 1;

int b = 2;

int *e = &b;

int *f;
f=&b;
```

```
printf("a = %d\nb = %d\ne = %d\n*e = %d\n&e = %d\n&b = %d\nf = %d\n*f = %d\n&f = %d\n"
                                    ,a,b,e,*e,&e,&b,f,*f,&f);
```

a = 1
b = 2
e = 1675541944
*e = 2
&e = 1675541936
&b = 1675541944
f = 1675541944
*f = 2
&f = 1675541928

a = 1
b = 2
e = 1675541944
*e = 2
&e = 1675541936

&b = 1675541944
f = 1675541944
*f = 2
&f = 1675541928

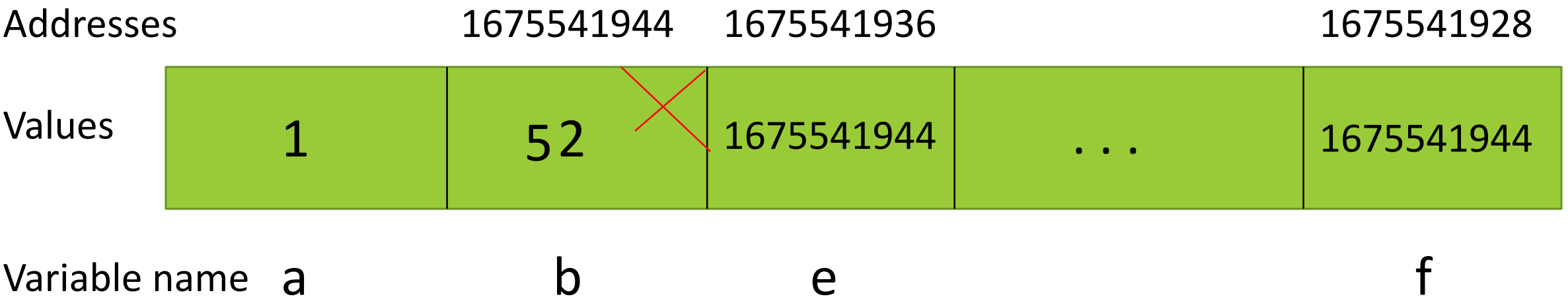


*f = 5

What will happen?!

```
root@kali:~# ./c
b = 5
e = 0x7ffe86738f18
*e = 5
&e = 0x7ffe86738f10
f = 0x7ffe86738f18
 *f = 5
&f = 0x7ffe86738f08
```

Notice that e, f, &f, &e contains memory addresses to view this address we used %p instead of %d in printf



C Pointer Syntax

Lets take a look at this example.

```c
#include <stdio.h>
int sum (int*, int*);

int main(){

        int a = 1;

        int b = 2;
        printf("%d\n",sum(&a,&b));
        return 0;
}
int sum(int *a,int *b){
        return *a + *b;
}
```

Pass the address of each variable to the function.

Function that takes 2 pointers as arguments.

Dereference each pointer to get the stored value at that memory location and compute their sum.

C Parameters Passing

1. By value:
   1. A duplicate copy of the argument is created and passed to the called function.
   2. Any update of the variable inside the function wont affect the original variable.
2. By reference
   1. The address of the variable is passed to the called function.
   2. Any update of the variable inside the function will affect the original value since the value is directly changed in its exact memory location.

By Value

```
int sum(int a,int b)
```

By Reference

```
int sum(int *a,int *b)
```

Exercise

1. Write a C program that reads two integers from the user and swaps them in the main.

2. Write a C program that reads 2 integers in the main and swaps them using a function swap()

```c
#include <stdio.h>

int main(){
        int i,j;
        scanf("%d%d",&i,&j);
        printf("Originally i = %d, j = %d\n",i,j);
        int tmp = i;
        i=j;
        j=tmp;
        printf("Now i = %d, j = %d\n",i,j);
        return 0;
}
```

Exercise 1 - Solution

```c
#include <stdio.h>
void swap(int *,int *);
int main(){
        int i,j;
        scanf("%d%d",&i,&j);
        printf("Originally i = %d, j = %d\n",i,j);
        swap (&i ,&j);
        printf("Now i = %d, j = %d\n",i,j);
        return 0;
}
void swap (int *a, int *b){
        int temp = *a;
        *a = *b;
        *b = temp;
}
```

Exercise 2 - Solution

Arrays

Arrays in C are composed of a particular type, laid out in memory in a repeating pattern. Array elements are accessed by stepping forward in memory from the base of the array by a multiple of the element size.

```c
#include <stdio.h>

int main(){
        char A[4] = {'a','b','c','\0'};
        int i = 0;

        A[0] = 'A';
        printf("%c\n",*(A+2));
        for(i = 0; i<4 ; i++)
                printf("%c\n",A[i]);
        return 0;

}
```

Brackets specifies the count of elements. Elements are set in the braces.

Access element at index 0 in array

Print element at index 2 using pointer notation. A evaluates to the address of the first element in the array.

For loops that iterates over the elements.

Exercise

Write a program that reads an integer n (1 < n <10), then fills an array of size n. Your program should compute the average of the n numbers.

```c
#include <stdio.h>

int main(){
        int array[10];
        int n;
        printf("Enter n: \n");
        scanf("%d",&n);
        int i = 0, sum = 0;
        for (i=0;i<n;i++)
        {
                printf("%d - Enter a number: \n",i);
                scanf("%d",&array[i]);
                sum +=array[i];
        }
        printf("Average = %f\n",(double) sum/n);
        return 0;
}
```

Exercise - Solution

```c
#include <stdio.h>
#include <stdlib.h>

int main(){

        int *vec;
        vec = malloc(sizeof(int)*3);
        *(vec) = 1;
        *(vec + 1) = 2;
        *(vec + 2) = 3;
        printf("vec[2]=%d\n", *(vec + 2));
        free(vec);
        return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>

int main(){

        int vec[3];
        vec[0] = 1;
        vec[1] = 2;
        vec[2] = 3;
        printf("vec[2]=%d\n", vec[2]);
        return 0;
}
```

Same function, different notations